

Name : Hasini

hasiniakkireddy@gmail.com

Role: AI/ML Intern

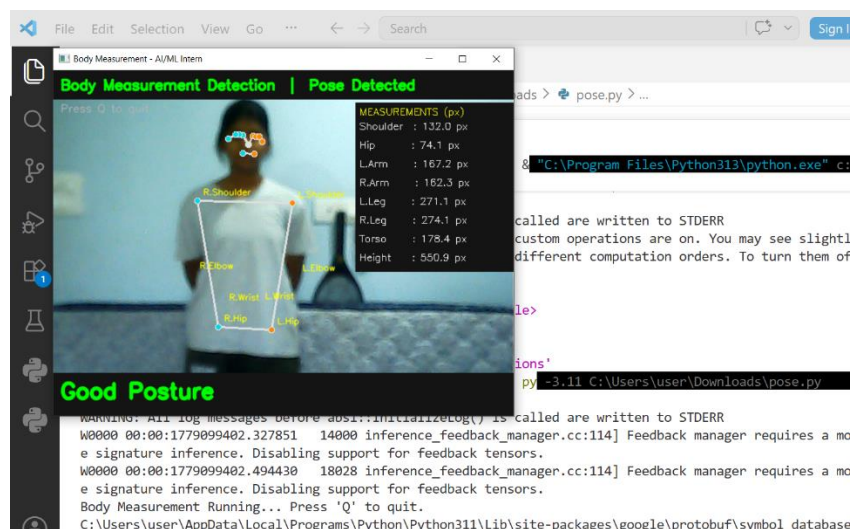
Day 6

Task 1: Improve Body Measurement Accuracy

I ran the improved body measurement detection code using MediaPipe and OpenCV. The system successfully detected the body pose in real time through the webcam and displayed 8 body measurements simultaneously on screen along with posture analysis.

What I Observed

- The system detected the pose successfully and showed Pose Detected status in green
- All 8 measurements updated in real time as the body moved
- The shoulder and arm landmarks were clearly visible on screen.
- Good Posture was detected correctly as both shoulders were level and body was aligned
- Left and right arm measurements showed a slight difference of 33.2 px which is normal due to camera angle
- Left and right leg measurements were close to each other confirming stable detection



Task 3 — Cloth Segmentation AI Research

What is Cloth Segmentation

Cloth segmentation is the process where AI separates the clothing region from the rest of the image. It identifies exactly which pixels in an image belong to a garment like a shirt, pants or dress and separates them from the person's skin, hair and background. The output is a clean clothing cutout that can be used in virtual try-on systems.

Segmentation Mask

A segmentation mask is a black and white image of the same size as the original image where:

- **White pixels (value = 1)** represent the clothing region
- **Black pixels (value = 0)** represent everything else like skin, hair and background

Think of it like a stencil. The white area is the clothing and the black area is everything that should be ignored.

Original Image + Segmentation Mask = Clothing Cutout

White pixel → Keep this pixel (clothing)

Black pixel → Remove this pixel (background/skin)

Image Mask

An image mask is the same as a segmentation mask but applied directly on the original image to produce the final output. When the mask is multiplied with the original image only the clothing pixels remain and everything else becomes transparent or black.

Output = Original_Image × Mask

Where:

Mask pixel = 1 → clothing pixel is kept

Mask pixel = 0 → pixel is removed

Cloth Segmentation Workflow

- **Step 1 — Input:** A person photo or clothing image is given as input
- **Step 2 — Preprocessing:** Image is resized and converted to the format the model needs
- **Step 3 — Model Prediction:** An AI model scans every pixel and predicts whether it belongs to clothing or not
- **Step 4 — Mask Generation:** The model outputs a segmentation mask showing clothing regions in white and everything else in black
- **Step 5 — Mask Application:** The mask is applied on the original image to extract only the clothing
- **Step 6 — Output:** A clean clothing cutout with transparent background is produced

MediaPipe Selfie Segmentation — separates full person from background, simpler but useful as a first step

Why Cloth Segmentation is Important in Virtual Try-On

In virtual try-on the system needs to:

- Remove the original clothing the person is wearing
- Place new clothing on the body

Without cloth segmentation the AI cannot identify where the clothing ends and the skin begins. The segmentation mask tells the system exactly which pixels to replace with the new garment. It is the most critical step between pose detection and final clothing overlay.



Task 2 — Pose Detection Testing on Image Files Report

Pose detection was tested on existing photos stored in local files instead of a live webcam. A photo saved on the laptop was given as input to the MediaPipe model and the skeleton with all body landmarks was drawn on that image as output.

How It Works

- An existing photo is loaded from the Downloads folder using OpenCV
- The image is converted to RGB format for MediaPipe to process
- MediaPipe detects all body landmarks from the photo
- Skeleton lines and landmark labels are drawn on the image
- The final output image is saved as a new file and displayed on screen

What I Observed

- The model successfully read the photo from the file
- Body landmarks were detected on the static image
- Skeleton was drawn connecting shoulders, elbows, wrists, hips, knees and ankles

Does Lighting Affect Accuracy?

Yes, lighting affects accuracy a lot.

- Poor lighting makes the image dark and the model cannot clearly see body edges and landmarks
- Too much brightness washes out body details and the model gets confused
- Shadows on one side of the body cause some landmarks to be missed or placed wrongly
- Best results come from standing in even front facing light with no strong shadows

Is Video Harder Than Image?

Yes, video is harder because:

- Video has 30 frames per second so the model must process each frame very fast
- Fast movement causes motion blur making landmarks harder to detect

- Landmarks must stay smooth and consistent across all frames which is more complex than detecting one single photo
- Video needs more computer power than a single image

Output:



Task 4 — Basic Recommendation Dataset Report

I created a basic clothing recommendation dataset manually in Microsoft Excel. The dataset contains 6 users with 5 columns representing their body type, occasion, style preference and recommended outfit.

Dataset Structure

Column

User

Gender

Occasion

Style

Size

Recommended Outfit

Dataset :

C13									
	A	B	C	D	E	F	G	H	
1	User	Gender	Occasion	Style	Size	Recommended			
2	U01	Male	Formal	Western	L	Formal Shirt,Pants			
3	U02	Male	Party	Ethnic	S	Kurta			
4	U03	Female	Party	Ethnic	M	Straight Kurta , Churidar			
5	U04	Male	Casual	Western	XL	T-Shirt, Jeans			
6	U05	Female	Sports	Western	S	Tracks			
7									
8									
9									
10									

Why This Dataset is Important

This dataset acts as the foundation for building an AI recommendation model. In future tasks this data will be used to train a machine learning model that learns patterns from user inputs and automatically predicts the most suitable outfit. Instead of using manual if-else conditions the model will learn from this data and give smarter recommendations as more user records are added.

Task 5 — Backend API Integration Report

A simple backend API was created using Flask in Python. The API takes user inputs like body type, occasion and style preference and returns a clothing recommendation as output. This connects the AI recommendation logic to a backend service that can be accessed by any frontend application.

What is Flask

Flask is a lightweight Python web framework used to build APIs and web applications. It allows Python code to be accessed over the internet or local network through HTTP requests making it easy to connect AI models with websites and mobile apps.

How the API Works

- The API runs on local server at `http://127.0.0.1:5000`
- It has two routes:
 - GET / — confirms the API is running
 - POST /recommend — takes user input and returns outfit recommendation
- User sends body type, occasion and style as input in JSON format
- API processes the input through the recommendation logic
- API returns the recommended outfit and fit type as JSON response

API Input and Output

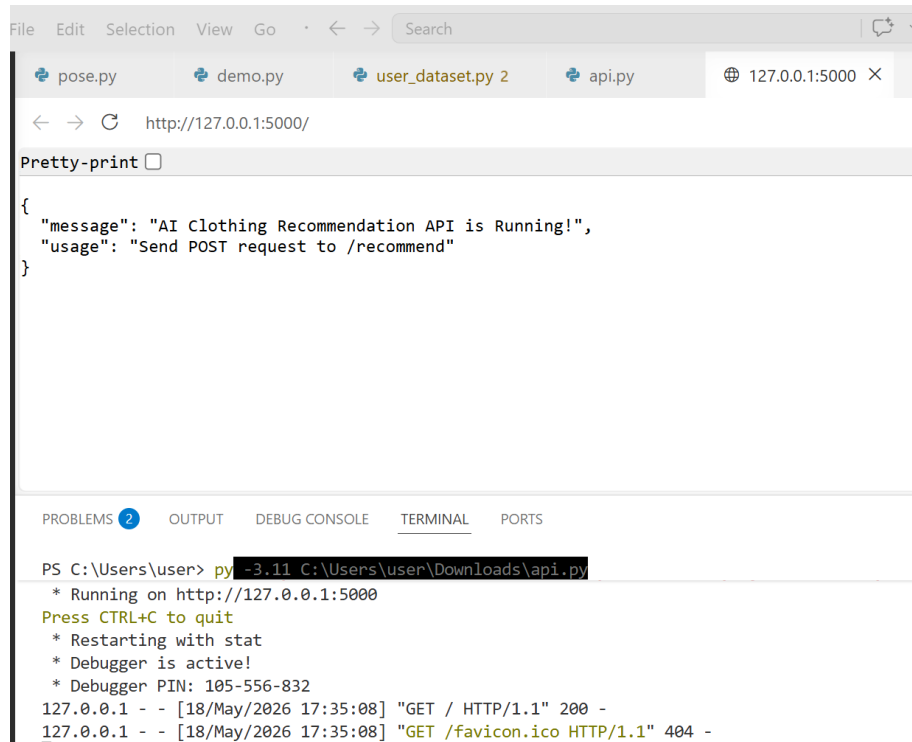
Input sent to API:

```
json
{
  "body_type": "slim",
  "occasion": "formal",
  "style": "western"
}
```

Output received from API:

```
json
{
  "status": "success",
  "recommendation": {
```

```
"outfit": "Blazer + Formal Shirt + Trousers",  
"fit": "Slim Fit"  
}  
}
```



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/`. The browser's developer tools are open, showing a JSON response in the console:

```
{  
  "message": "AI Clothing Recommendation API is Running!",  
  "usage": "Send POST request to /recommend"  
}
```

Below the browser window, a terminal window is open, showing the command `python -3.11 C:\Users\user\Downloads\api.py` being executed. The terminal output indicates that the API is running on `http://127.0.0.1:5000` and provides instructions to press `CTRL+C` to quit. It also shows the status of the server, including the restart command `stat` and the debugger PIN `105-556-832`. The terminal also displays two log entries:

```
127.0.0.1 - - [18/May/2026 17:35:08] "GET / HTTP/1.1" 200 -  
127.0.0.1 - - [18/May/2026 17:35:08] "GET /favicon.ico HTTP/1.1" 404 -
```